# TP 2 : Architecture par Sous-Systèmes

**Master Programmation Jeu Vidéo : Architecture C++ & Raylib**

> Modalités du TP
>
> — **Évaluation** : orale, en classe, le jeudi 22 octobre 2026
> — **Pondération** : 20% de la note finale
> — **Langage & outils** : C++17 ou C++20, CMake (FetchContent pour Raylib 5.0), Raylib
> — **Modalité** : Individuel (binômes et partage de code formellement interdits)
> — **Point de départ** : votre Space Invaders du TP 1, qui doit rester entièrement fonctionnel (y compris l'audio)

## 1 Objectif

Le TP 1 a produit un prototype jouable : quelques structures plates, des fonctions `Update` et `Draw`, et une boucle principale qui orchestre le tout. Ce découpage suffit pour un prototype, mais il ne résiste pas à la croissance d'un vrai projet : l'ordre d'initialisation est implicite, les ressources sont libérées « à la main » dans `main()`, chaque nouveau système ajoute des paramètres à toutes les fonctions, et rien ne garantit qu'un objet survive (ou meure) au bon moment.

L'objectif de ce TP est de **refactoriser votre jeu vers une architecture par sous-systèmes inspirée d'Unreal Engine** (`UEngineSubsystem`, `UWorldSubsystem`, `ULocalPlayerSubsystem`). Chaque système important devient un objet à instance unique **dans la portée de son propriétaire**, dont la durée de vie est automatiquement liée à celle de l'objet hiérarchique qui le possède :

— l'**Engine** crée et détruit les sous-systèmes moteur ;
— le **World** crée et détruit les sous-systèmes de monde ;
— chaque **LocalPlayer** crée et détruit ses propres sous-systèmes de joueur.

À l'issue du TP, votre jeu doit être **maintenable** (chaque responsabilité a un seul propriétaire), **extensible** (ajouter un système ne modifie pas `main()` ni les systèmes existants) et **prêt pour la production** (cycle de vie déterministe, zéro fuite, comportement observable et testable).

## 2 Modèle de référence

Le tableau suivant fixe la hiérarchie attendue. Un objet de niveau inférieur ne peut jamais survivre à son propriétaire.

| Niveau | Propriétaire | Durée de vie | Exemples de sous-systèmes |
|---|---|---|---|
| Moteur | `Engine` | Toute l'exécution du processus | Fenêtre/Rendu, Audio, Ressources, Entrées brutes, Sauvegarde |
| Joueur local | `LocalPlayer` (possédé par l'`Engine`) | De la connexion à la déconnexion d'un joueur local, **indépendante des changements de monde** | Correspondance touches/actions, profil et statistiques cumulées |
| Monde | `World` (un seul actif, possédé par l'`Engine`) | D'un chargement de monde à son déchargement | Pool de projectiles, Flotte, Collisions, Score de la partie, Règles du jeu |

Pourquoi pas de singletons classiques ?

Un singleton global (`static Foo& Get()`) est créé « quelque part » à la première utilisation et détruit « quelque part » après `main()`, dans un ordre que vous ne contrôlez pas. Un sous-système offre la même commodité (une instance unique, accessible facilement) mais **dans une portée explicite** : il naît quand son propriétaire naît, meurt quand son propriétaire meurt, et peut être recréé proprement (nouvelle partie, nouveau joueur). C'est cette garantie qui est évaluée.

# 3 Spécifications fonctionnelles

## 3.1 Jalon 1 : Socle des sous-systèmes

Concevoir l'infrastructure générique, indépendante du gameplay :

— **Classe de base** `Subsystem` exposant au minimum les points d'entrée suivants :

```
class Subsystem {
public:
    virtual ~Subsystem() = default;
    virtual bool ShouldCreateSubsystem(const void* outer) const { return true; }
    virtual void Initialize(SubsystemCollection& collection) {}
    virtual void Deinitialize() {}
};

class EngineSubsystem      : public Subsystem { /* Engine&      GetEngine() const; */ };
class WorldSubsystem       : public Subsystem { /* World&       GetWorld() const; */ };
class LocalPlayerSubsystem : public Subsystem { /* LocalPlayer& GetLocalPlayer() const;
```

Vous êtes libres d'adapter les signatures (par exemple typer `outer`), à condition de justifier vos choix à l'oral.

— **Enregistrement** : l'ajout d'un nouveau sous-système ne doit nécessiter **aucune modification** du code de l'`Engine`, du `World` ou de `main()`. Approche recommandée : enregistrement statique par macro ou par registre de fabriques (`REGISTER_WORLD_SUBSYSTEM(FleetSubsystem)`).

— **Création conditionnelle** : `ShouldCreateSubsystem` permet de ne pas instancier un sous-système selon le contexte (ex. : pas de `FleetSubsystem` dans le monde du menu principal).

### 3.2 Jalon 2 : Engine et sous-systèmes moteur

- Une classe `Engine` qui possède la boucle principale, la collection de sous-systèmes moteur, le `World` actif et la liste des `LocalPlayer`. Le `main()` final doit tenir en quelques lignes (création de l'`Engine`, `Run()`, fin).
- Sous-systèmes moteur **recommandés** :
  - `WindowSubsystem` : `InitWindow` / `CloseWindow`, cible de rendu, configuration de la fenêtre ;
  - `AudioSubsystem` : `InitAudioDevice` / `CloseAudioDevice`, lecture des sons. Il doit déclarer une dépendance au `WindowSubsystem` afin que l'ordre inverse de désinitialisation garantisse naturellement la règle du TP 1 (`CloseAudioDevice()` avant `CloseWindow()`), **sans code d'ordonnancement manuel** ;
  - `AssetSubsystem` : chargement unique des sons et textures à l'initialisation, accès par identifiant (enum ou handle), libération à la désinitialisation. `LoadSound()` reste interdit dans la boucle ;
  - `InputSubsystem` : seule classe autorisée à appeler les fonctions d'entrée de Raylib (`IsKeyDown`, `IsKeyPressed`, etc.). Elle échantillonne l'état brut du clavier une fois par frame ;
  - `SaveSubsystem` : charge le meilleur score depuis un fichier à l'initialisation et l'écrit à la désinitialisation. Le meilleur score doit **survivre aux changements de monde et aux redémarrages du jeu**.

### 3.3 Jalon 3 : World, tick et transitions de monde

- Une classe `World` qui possède la collection de sous-systèmes de monde ainsi que les données de simulation (pools de projectiles, grille d'aliens, canons des joueurs).
- Sous-systèmes de monde **recommandés** :
  - `ProjectileSubsystem` : pool fixe de projectiles (repris du TP 1) ;
  - `FleetSubsystem` : grille d'aliens, machine à états de la flotte, accélération ;
  - `CollisionSubsystem` : tests AABB et résolution des impacts ;
  - `ScoreSubsystem` : score de la partie en cours, attribué au joueur auteur du tir ;
  - `GameRulesSubsystem` : conditions de victoire et de défaite, déclenchement des transitions.

### 3.4 Jalon 4 : Joueurs locaux

- Une classe `LocalPlayer` représente une **personne** devant l'écran, distincte du **canon** qu'elle contrôle dans le monde. Le `LocalPlayer` survit aux transitions de monde ; le canon, lui, appartient au `World`.
- Sous-systèmes de joueur **recommandés** :
  - `PlayerInputSubsystem` : table de correspondance touches vers actions (`MoveLeft`, `MoveRight`, `Fire`) propre à chaque joueur, à la manière d'Enhanced Input. La logique de gameplay ne manipule **que des actions**, jamais des touches ;
  - `PlayerStatsSubsystem` : statistiques cumulées du joueur sur la session (aliens détruits, parties jouées), conservées entre les mondes.

### 3.5 Jalon 5 : Communication découplée

Les sous-systèmes ne doivent pas former un graphe d'appels directs enchevêtré :

- Implémenter un mécanisme de **délégués multicast** (ou un bus d'événements) pour les événements de gameplay, au minimum : `OnShotFired`, `OnAlienDestroyed`, `OnFleetStep`, `OnGameEnded`.
- Le `AudioSubsystem` **s'abonne** à ces événements ; aucun sous-système de gameplay ne doit l'appeler directement.

— Les abonnements effectués par un sous-système de monde doivent être **retirés lors de sa désinitialisation**. Après une transition de monde, aucun délégué ne doit pointer vers un objet détruit.
— Le mécanisme doit respecter la discipline mémoire : capacité fixe, liaisons effectuées à l'initialisation, aucune allocation lors de la diffusion (attention : `std::function` peut allouer).

### 3.6 Jalon 6 : Observabilité

— **Trace du cycle de vie** : chaque `Initialize` et `Deinitialize` produit une ligne de log indiquant le niveau, le nom du sous-système et le propriétaire. Cette trace sera utilisée à l'oral pour démontrer l'ordre d'initialisation et de destruction.
— **Superposition de débogage** : une touche (ex. : `F1`) affiche à l'écran la liste des sous-systèmes actifs par niveau (moteur, joueurs, monde). Vous pouvez afficher cela dans le log ou pour 10 points bonus utiliser ImGUI.

## 4 Contraintes techniques & Rendu

**!** Règles et Contraintes

— **Héritage du TP 1** : toutes les contraintes du TP 1 restent en vigueur (passages par référence et référence constante, pools à taille fixe, cycle de vie audio, CMake autonome avec FetchContent).
— **Zéro allocation en régime établi** : aucune allocation sur le tas pendant une frame ordinaire. Les allocations sont tolérées **uniquement** lors des événements de cycle de vie : initialisation de l'`Engine`, chargement ou déchargement d'un `World`, ajout ou retrait d'un `LocalPlayer`.
— **Aucun singleton sauvage** : pas de variable globale mutable ni de `static T& Get()`. Une seule exception tolérée : un point d'accès global à l'`Engine` (équivalent de `GEngine`), à justifier.
— **Propriété explicite** : toute ressource (son, texture, fichier, abonnement à un délégué) a exactement un propriétaire, libère ce qu'elle acquiert, et ne survit jamais à son propriétaire.
— **Raylib confiné** : les appels Raylib de fenêtre, d'entrée et d'audio sont confinés à leurs sous-systèmes moteur respectifs.